

## Grouping and Pivoting — Instructor Notes

Class 12 · group, pivot, confounding, data provenance · handout + notebook

**Read before teaching: target in this file is inverted.** target = 1 means **no** heart disease. I matched all 1,025 rows to the original UCI Cleveland file (processed.cleveland.data, now saved in data/) on age, sex, blood pressure, cholesterol, max heart rate and ST depression. Every row matched, and the mapping is exact:

|                   | UCI num = 0<br>(no disease) | UCI num > 0<br>(disease) |
|-------------------|-----------------------------|--------------------------|
| Kaggle target = 1 | 164 patients                | 0                        |
| Kaggle target = 0 | 0                           | 138 patients             |

What this changes:

- **The “cholesterol paradox” doesn’t exist.** With correct labels the disease group has the *higher* cholesterol (UCI: mean 251.5 vs. 242.6, median 249 vs. 234.5). That is the expected direction.
- **Class 09 is affected too.** The 72.4% that notes call  $P(\text{disease}|\text{female})$  is really  $P(\text{no disease}|\text{female})$ . In the original data it’s 25.8% for women and 55.3% for men.
- Part 4’s arithmetic still holds on the file as posted, and so does Part 4’s lesson about controlling for a variable. Its *clinical* story is backwards.

**Options:** (a) teach it as written and reveal the inversion as the payoff of Part 5, “Do you trust this table?” (the strongest version, and it needs only a closing slide; see *Suggested closer*); (b) recode target in the notebook’s load cell and rewrite Part 4 as a masking example; (c) teach as written and correct it next class. I recommend (a). Either way, Class 09 needs a correction.

**The idea.** group and pivot compress a table into something legible, and every compression hides something. The activity runs from convenience (Parts 1–3) to what the summaries hide: a confounder that makes a gap look smaller than it is (Part 4), and a file whose documentation, row count, and, it turns out, outcome label don’t mean what they say (Part 5).

**What students produce.** Two versions of the same comparison (long way, one line), a pivot sketch, a four-cell table of means, and the metadata and duplication findings. **What this does not claim to teach:** causal inference, formal Simpson’s paradox, or anything about cholesterol as a risk factor. The data can’t support that last one (see the box above).

**Materials.** Handout, notebook, and data/heart.csv in the same folder as the notebook. The Class12 folder had no data folder; I copied heart.csv in from Class09. Make sure it’s uploaded next to the notebook on the hub, or the load cell fails.

### Timing (50 min)

| Segment | Min | What happens                                                        |
|---------|-----|---------------------------------------------------------------------|
| Part 1  | 8   | The long way with where. Count lines of code.                       |
| Part 2  | 14  | Run the worked group cells, redo 1.1 in one line, value_range, 2.2. |
| Part 3  | 10  | Sketch before running the pivot; the zero-filled pivot and 3.2.     |

|        |    |                                                                    |
|--------|----|--------------------------------------------------------------------|
| Part 4 | 12 | Histogram, 4.1, commit to 4.2, then the four-cell table, 4.4, 4.5. |
| Part 5 | 6  | thal, ca, distinct rows, 5.3. Closer if you choose option (a).     |

**Watch out.** The 50-minute budget is an estimate. Part 2 has the most runnable-but-skippable material (the optional step-by-step cell, the value\_range demo); trim there first. Protect Part 5. It's short, and it's the payoff.

## The numbers

All from heart.csv as posted (1,025 rows), in the local ds313 environment.

|                                  |                                                                    |
|----------------------------------|--------------------------------------------------------------------|
| Median age                       | 56 → younger ( $\leq 56$ ) 545 rows, older 480 rows                |
| Mean chol, younger / older       | 239.5 / 253.3                                                      |
| Mean resting BP, younger / older | 127.5 / 136.3                                                      |
| Mean chol, target 0 / 1          | 251.3 / 241.0 (medians 249 / 234)                                  |
| Mean chol by sex × target        | women 276.7 / 255.6; men 246.0 / 229.9                             |
| Women / men                      | 312 / 713 rows; mean chol 261.5 / 239.2; target mean 0.724 / 0.421 |
| thal values found                | 0, 1, 2, 3 (counts 7, 64, 544, 410)                                |
| ca values found                  | 0–4 (counts 578, 226, 134, 69, 18)                                 |
| Distinct rows                    | 302; each appears 3 (187 rows), 4 (114) or 8 (1) times             |

## Part 1. The Long Way

(≈ 8 min)

**1.1** Median age 56. At-or-below vs. above: chol 239.5 vs. 253.3, resting BP 127.5 vs. 136.3. A team that splits below vs. at-or-above gets 238.0 vs. 253.8 and 126.9 vs. 136.2 (506 / 519 rows). Both are fine; the notebook specifies at or below. Older patients are higher on both.

**1.2** Typically 6–10 lines: compute the median, two wheres, four np.means.

## Part 2. group()

(≈ 14 min)

**2.1** One line, one call:

```
heart.with_column('older', heart.column('age') > np.median(heart.column('age'))).group('older',
np.mean).select('older', 'chol mean', 'trestbps mean')
```

Same numbers as 1.1 (False = younger). What group does that a boolean can't: a boolean splits the table in two, one subset per where. group splits on *every distinct value at once* (41 ages in cell 9) and applies the collect function to every other column in the same call. Strong answers say both: many groups, and all columns.

**Note.** group also accepts the boolean array directly, without adding a column: heart.group(heart.column('age') > 56, np.mean). The grouping column comes back labeled group. Either

version is fine. Predictable snag: `group(..., np.mean)` averages *every* column, so teams hunt through 13 columns for chol mean unless they select.

**2.2** Defining range works silently; nothing happens until cell 45, where for `i` in `range(10)` raises `TypeError: 'int' object is not iterable`. The traceback points at `max(x)` inside their own range function, not at the loop. Why it's worse: the error shows up far from the cause, in code that is correct, with a message about the wrong thing, and it survives “restart and run all” because cell 20 still runs before cell 45. A bug that fails at cell 20 would point straight at the problem.

### Part 3. `pivot()`

(≈ 10 min)

**3.1** `pivot(columns, rows)`: slope values become **columns** (0, 1, 2), target values become **rows** (0, 1). Two rows, a target label column plus three count columns. Each cell is the number of patients with that target and that slope:

| target | 0  | 1   | 2   |
|--------|----|-----|-----|
| 0      | 46 | 324 | 129 |
| 1      | 28 | 158 | 340 |

The usual wrong sketch swaps rows and columns: `pivot` takes the columns argument *first*. The cells sum to 1,025, which is a quick check.

**3.2** In the age × target pivot, 7 of 82 cells are empty and were filled with 0: no target 0 patients at ages 29, 34, 37, 71, 74, 76, and no target 1 patient at 77. Filtering them leaves 34 ages. Examples of the same thing elsewhere: sensors that log 0 or -9999 for “no reading”; spreadsheet formulas that treat a blank as 0; lab results below the detection limit entered as 0; survey non-responses coded 99. Better: leave it missing (nan) so it's excluded from averages and plots, or fail loudly. The best answer is in Part 5: `thal = 0` and `ca = 4` in this very file are missing values that were written as numbers (see 5.1). If a team gets there early, hold it.

### Part 4. The Cholesterol Paradox

(≈ 12 min)

**4.1** `heart.group('target', np.mean)` and `np.median`: mean 251.3 (target 0) vs. 241.0 (target 1); median 249 vs. 234. The target 1 group is about 10 mg/dl lower.

**4.2** Anything, in ink. Collect a few on the board: treatment (statins), selection (referred patients), composition (sex, age), “cholesterol doesn't matter,” “the labels are wrong.” Keep the last one visible if anyone says it. They're right.

**4.3** Mean cholesterol:

|          | target 0 | target 1 | gap   |
|----------|----------|----------|-------|
| women    | 276.7    | 255.6    | -21.1 |
| men      | 246.0    | 229.9    | -16.1 |
| everyone | 251.3    | 241.0    | -10.3 |

**4.4 WIDENS.** The gap is about 21 for women and 16 for men, against 10 overall. Women have higher cholesterol *and* make up most of the target 1 group (226 of 526, versus 86 of 499 in target 0), so the aggregate target 1 mean is propped up by women's cholesterol. Mixing the sexes *hides* part of the gap. Composition was real, but it worked against the paradox, not for it. Controlling for sex made the difference bigger. That's the transferable lesson: a confounder can mask a difference as easily as manufacture one.

**Note.** The notebook's setup (cell 32) frames composition as a testable prediction: if the gap comes from who is in each group, it should shrink when you compare within sex. Cell 33 shows the group makeup first. In the file, the target 0 group is 83% men (413 of 499), so the prediction fails, which is the point.

**4.5 Treatment:** a column for medication, or at least the date of the cholesterol measurement relative to diagnosis. **Selection:** a comparison group that wasn't referred, i.e. people sampled from the general population. Stronger answers note that no column can be added *to this file* to fix selection; it's a property of how the rows got there.

**Note.** With correct labels, neither 4.5 story is needed: the disease group has the higher cholesterol. Both are still real concerns with clinical data, and good ones to discuss. Just don't let the class leave thinking they explained this particular result.

## Part 5. Do You Trust This Table?

(≈ 6 min)

**5.1** thal: documented 0/1/2 (normal, fixed, reversible); found **0–3**. ca: documented 0–3; found **0–4**. Against the UCI original: thal 2 = normal, 1 = fixed defect, 3 = reversible defect, and **0 = missing** (the 2 UCI patients recorded as "?"). ca 4 = **missing** (the 4 "?" patients). So the documented coding is wrong, and the most common value, 2, which the documentation calls "reversible defect," actually means normal. cp and slope are recoded from UCI too, and the metadata doesn't give either code.

**5.2** 1,025 rows, 302 distinct. Each patient was copied 3 or 4 times to inflate 303 patients to 1,025 rows. More precisely, 302 of the 303 UCI patients appear. One patient (60-year-old woman, chol 305) is missing and one (38-year-old man, chol 175) appears 8 times. We can see what was done, not why.

**5.3** It makes chance variation look smaller than it is, so results look more certain than they are. Standard errors shrink by  $\sqrt{1025/302} \approx 1.84$ . Intervals come out about 46% too narrow, and differences that are really noise can look significant. Direction matters: the error is always toward overconfidence. (Class 19's bootstrap resamples rows, so it would inherit exactly this problem.)

**Note. Suggested closer for option (a).** After 5.3, put up the matched-rows table from the box on page 1: "Every row in this file has a twin in the original study. In all 302, target = 1 is the patients *without* disease." Then ask what that does to 4.1 and to Class 09's 72%. Two minutes, and it makes Part 5's question concrete: they trusted a column label for two classes.

## If You Finish Early

**E1** 41 points, one per distinct age; median cholesterol vs. age has  $r \approx 0.57$ . It's evidence that, *among these referred patients*, older age groups have higher median cholesterol. It's *not* evidence that a person's cholesterol rises as they age (these are different people at one time, not one person over time), nor for the general population (a referral sample). It also treats every age as one equal point: some ages rest on a single distinct patient (4 ages), others on 19, and the 1,025 rows inflate every count by about  $3.4\times$ . Strong answers name at least two of: cross-sectional vs. longitudinal, population, unequal support per point.

**E2** Raw rows only: the duplication (5.2) and the undocumented codes (5.1). Any summary hides both, and the inverted target was found only by matching individual rows. Aggregation only: the age trend, the within-sex gap in 4.4, the 7 empty pivot cells. None of those is visible by reading 1,025 rows. Good answers note that the zeros in 3.2 were an *artifact* of aggregation.

## Appendix A: solution code (instructor only)

```
# 1.1, the long way
median_age = np.median(heart.column('age'))
younger = heart.where('age', are.below_or_equal_to(median_age))
older = heart.where('age', are.above(median_age))
print(np.mean(younger.column('chol')), np.mean(older.column('chol')))
print(np.mean(younger.column('trestbps')), np.mean(older.column('trestbps')))

# 4.1
heart.group('target', np.mean).select('target', 'chol mean')
heart.group('target', np.median).select('target', 'chol median')

# 4.3
heart.select('sex', 'target', 'chol').group(['sex', 'target'], np.mean)
```

## Appendix B: checking target against the original study

data/uci\_processed\_cleveland.data is the UCI file (no header; column 14, num, is 0 for no disease and 1–4 for disease). This counts how target lines up with it, matching patients on six measurements. It prints {(0, 1): 138, (1, 0): 164}: target 1 is always the UCI “no disease.”

```
import csv

uci = {}
for line in open('data/uci_processed_cleveland.data'):
    f = line.strip().split(',')
    key = (float(f[0]), float(f[1]), float(f[3]), float(f[4]), float(f[7]), float(f[9]))
    uci[key] = int(float(f[13]) > 0)    # 1 = disease in the original study

pairs = {}
seen = []
for row in csv.DictReader(open('data/heart.csv')):
    key = (float(row['age']), float(row['sex']), float(row['trestbps']),
           float(row['chol']), float(row['thalach']), float(row['oldpeak']))
    if key not in seen:
        seen.append(key)
        pair = (row['target'], uci[key])
        pairs[pair] = pairs.get(pair, 0) + 1
print(pairs)
```